Graph-Structured Reinforcement Learning for Fair and Efficient CPU Process Scheduling

V. Saadhanaa
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
saadhanaa.2025@vitstudent.ac.in

Disha Vishal Chaudhari
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
disha.vishal2025@vitstudent.ac.in

Sankari M.
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
sankari.m@vit.ac.in

***Abstract*—However, FCFS, Round Robin, and CFS in Linux do not explicitly consider starvation prevention for low-priority processes. To address these challenges, we designed a scheduling framework—GraphSAGE + MaskablePPO—targeting both efficiency and fairness. At each step, the framework discards the existing graph and generates a new graph based on three types of relations between processes: contending for CPU, sharing the same priority level, and having a similar burst pattern lately. GraphSAGE [1] extracts features from the graph while a MaskablePPO agent [2] selects the next process to run. Moreover, we defined a reward function such that neglecting the request from an underprivileged process becomes increasingly costly after some threshold waiting time; hence, the agent learns by itself how to avoid starvation.**

We conducted our experiment for 500 seed runs using bootstrap confidence intervals. Compared to CFS [3], the average waiting time is reduced by 23.7 % and starvation by 24.4 pp on uniform workloads. With exponential bursts, the advantage in terms of throughput practically disappears; however, starvation is reduced by 11.5 pp. The most unexpected conclusion: without a graph, a flat-MLP agent cannot outperform CFS on realistic workloads. The relational structure appears to be the key to success of our RL solution.

***Index Terms*—process scheduling, starvation prevention, GraphSAGE, MaskablePPO, graph neural networks, reinforcement learning, CPU fairness, priority aging**

I. Introduction

CPU scheduling has been studied extensively, yet the core problem persists. SRTF is provably the best you can do for average wait—but it needs to know how long each burst will last before it happens, which no real scheduler can. So in practice we get heuristics. CFS [3] is arguably the strongest: it tracks a virtual runtime per process, always picking whichever one has used the least CPU time (weighted by nice value). CFS is fast, elegant, and surprisingly hard to beat. But it has a blind spot. Nothing in CFS says "this process has been sitting here too long, run it." Priority aging shows up in textbooks [4], [5], sure—but CFS does not actually use it.

Recent advances in deep reinforcement learning have demonstrated promising results for resource scheduling at the cluster and job levels—DeepRM [6] for cluster resource allocation, Decima [7] for Spark job DAGs, and RLScheduler [8] for HPC batch queues. However, these systems operate at a considerably coarser granularity than per-timeslice CPU dispatch. To the best of our knowledge, no prior work has combined a graph neural network with action masking and a fairness-aware reward for per-process OS-level scheduling.

This paper addresses that gap with three contributions:

1) A dynamic process graph reconstructed at every scheduling tick from observable process features, encoding contention, type similarity, and burst-history similarity relationships.
2) A GraphSAGE-based feature extractor integrated with MaskablePPO. GraphSAGE [1] is specifically suited to this setting, as it was designed for graphs whose topology changes between forward passes—processes arrive, block on I/O, resume, and terminate during execution.
3) A soft-priority aging reward that formalises the textbook concept of priority aging as an RL training signal, enabling the policy to learn anti-starvation behaviour from reward shaping alone.

II. Related Work

DeepRM [6] pioneered deep RL for scheduling through training a REINFORCE policy on a two-dimensional image-based representation of cluster resources. RLScheduler [8] extended DeepRM with per-job feature vectors and PPO to emulate HPC heuristics on six real-world workloads. Neither DeepRM nor RLScheduler works at the OS process level. Moreover, neither one takes graph structure into consideration nor does either include fairness in its reward design.

Decima [7] is the closest work which uses graph structure for scheduling: it runs a customised GNN on static Spark DAGs and achieves a 21 % reduction in average job completion time. However, Decima's graph is a fixed dependency DAG that gets changed only upon a new job arrival; we regenerate our graph at every tick. Decima has no action masking, and it does not address priorities. L2D [9] used GIN in manufacturing job-shop scheduling. A recent survey [10] concluded that GCN, GIN, and GAT dominated the scheduling literature—GraphSAGE has not previously appeared in any scheduling domain. Adimora et al. [11] ran Graph Attention Networks on 377K jobs in real supercomputers and argued that inductive GNNs are critical when queue lengths are always

varying. We aim to do the same, but at a much finer grain—individual timeslices rather than batch jobs.

Regarding masking, Huang and Ontañón [12] demonstrated that constraining invalid logits to $-\infty$ converges $57\times$ faster than reward penalty techniques. Lassoued et al. [13] applied policy-based RL with action masking to dynamic job-shop scheduling under uncertainty—the closest existing work to ours, but with no GNN and no CPU-level dispatch.

Regarding fairness, we found no RL scheduler which learns aging via its reward signal. Mamirov [14] uses aging as a modular fairness mechanism for GPU cluster scheduling; STUN [15] utilises tabular Q-learning for adjusting CFS parameters globally. Neither of them schedules individual processes.

III. System Design

A. Environment

The environment models a single-CPU system with up to eight processes running concurrently [16]. Each process switches between CPU bursts and I/O wait periods. The scheduler takes into account all information that a dispatcher in an actual operating system has access to—waiting time, process type, CPU utilisation, burst history—but cannot predict future burst durations, maintaining the causality constraint inherent in real scheduling.

Each process slot contributes 8 features to a flat 64-dimensional observation vector:

$$\mathbf{o}_i = \big[\, w_i,\ \tau_i,\ u_i,\ s_i,\ b_i^{\text{last}},\ \bar{b}_i,\ r_i,\ d_i \,\big]$$

representing waiting time, type, CPU used, schedule count, last and average burst length, ready flag, and blocked time, respectively. The action space is $\mathrm{Discrete}(N_{\max}{+}1)$: the agent picks one process for dispatching or idles. Invalid actions (selecting a terminated or I/O-blocked process) are masked to $-\infty$ prior to the softmax.

B. Reward Function

Every tick, the agent incurs a penalty for each ready process that it did *not* choose, proportional to the type of that process. In addition, when any process has been waiting longer than a threshold W^*, an extra penalty is triggered that grows linearly with the overshoot:

$$r_t = -\sum_{i \neq a_t,\ i \text{ ready}} \Big[\, w_{\tau_i} + \alpha \cdot \max\big(0,\ W_i - W^*\big) \Big] \qquad (1)$$

We fix $\alpha = 0.5$ and $W^* = 10$ for all experiments. The consequence is straightforward: the longer a process goes unserved, the more costly it becomes, and thus the agent learns to attend to it—not through a hand-written scheduling rule, but through the reward signal alone.

C. Dynamic Process Graph

The entire graph is discarded and reconstructed from scratch at every tick. Processes become nodes; there are three types of edges:

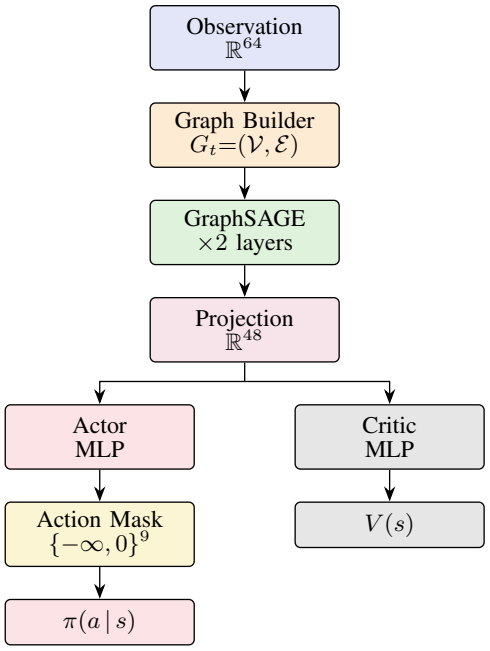


Fig. 1. End-to-end architecture of the proposed GraphSAGE + MaskablePPO framework. The process graph is reconstructed at every tick from observable features. Invalid actions are masked prior to the softmax.

1) **CPU Contention.** When two processes share the ready queue, they receive an edge. This is the most straightforward relationship—both are contending for CPU time right now.
2) **Type Similarity.** Sharing a priority class ($\tau_i = \tau_j$) results in an edge. The goal is to allow the policy to reason about groups of processes rather than just individuals.
3) **Burst-History Similarity.** If two processes have average burst lengths that differ by no more than one tick ($|\bar{b}_i - \bar{b}_j| \leq 1$), they receive an edge. This turned out to be quite relevant, as it gives the policy a rough sense of which processes behave alike, all without revealing any future burst information.

All edges include self-loops. No state carries over between ticks.

D. GraphSAGE Feature Extractor

Neighbourhood aggregation is performed via mean-pooling using two GraphSAGE layers across two rounds of message passing:

$$\mathbf{h}_i^{(1)} = \mathrm{ReLU}\Big(W_1\big[\,\mathbf{x}_i \,\|\, \overline{\mathbf{x}}_{\mathcal{N}(i)}\,\big]\Big) \qquad (2)$$

$$\mathbf{h}_i^{(2)} = \mathrm{ReLU}\Big(W_2\big[\,\mathbf{h}_i^{(1)} \,\|\, \overline{\mathbf{h}}_{\mathcal{N}(i)}^{(1)}\,\big]\Big) \qquad (3)$$

where $W_1 \in \mathbb{R}^{32\times 20}$ and $W_2 \in \mathbb{R}^{16\times 64}$. The resulting node embeddings are concatenated to form a 48-dimensional vector, which feeds a standard PPO actor-critic network with two hidden layers of 64 units each. Fig. 1 shows the complete architecture.

TABLE I
FULL BENCHMARK ACROSS 500 SEEDED EPISODES. † DENOTES ORACLE ACCESS TO HIDDEN FUTURE BURST LENGTHS (THEORETICAL BOUND, NOT A FAIR COMPARISON).

Scheduler	Uniform		Realistic	
	Avg Wait	Starv.	Avg Wait	Starv.
FCFS	23.91	88.5 %	34.56	88.4 %
Round Robin	24.70	89.1 %	30.93	86.5 %
Priority + Aging	24.78	90.4 %	31.79	87.9 %
MLFQ	24.85	90.2 %	31.76	87.5 %
CFS	24.31	87.3 %	28.33	79.2 %
Stride	24.30	87.3 %	28.35	79.3 %
Plain MaskablePPO	19.30	65.4 %	28.28	68.5 %
GraphSAGE + MaskablePPO	18.54	62.9 %	28.00	67.7 %
SRTF†	14.49	53.1 %	17.32	52.4 %

TABLE II
CONTRIBUTION DECOMPOSITION. Δ DENOTES PAIRED MEAN DIFFERENCE (NEGATIVE IS BETTER). ✓ MEANS THE 95 % CI EXCLUDES ZERO.

WL	Transition	Δ Avg	95 % CI	Gap	Sig.
Unif.	Classical→RL flat	-4.6	$[-5.0, -4.2]$	49%	✓
	RL flat→RL+Graph	-0.8	$[-1.0, -0.5]$	8%	✓
	RL+Graph→SRTF	-4.1	$[-4.3, -3.8]$	43%	✓
Real.	Classical→RL flat	$+0.6$	$[-0.3, +1.4]$	n/a	×
	RL flat→RL+Graph	-0.7	$[-1.4, -0.1]$	7%	✓
	RL+Graph→SRTF	-10.9	$[-11.9, -10.0]$	99%	✓

IV. EXPERIMENTAL SETUP

Uniform workloads. The durations of CPU and I/O bursts are sampled from $\mathcal{U}\{1, 5\}$, while burst counts come from $\mathcal{U}\{2, 4\}$. Process types are assigned uniformly at random. Seeds 10 000–10 499 (500 episodes).

Realistic workloads. Exponential distributions with type-dependent means based on the Coffman–Kleinrock model [17] and standard OS textbook parameters [5]: SYSTEM (μ_{cpu}=2.5, μ_{io}=2.0), INTERACTIVE (μ_{cpu}=4.0, μ_{io}=5.0), BACKGROUND (μ_{cpu}=8.0, μ_{io}=2.0). Type distribution: 20/50/30 %. Seeds 20 000–20 499 (500 episodes).

Baselines. Six classical schedulers are evaluated: FCFS, Round Robin (q=2), Priority + Aging, MLFQ [4] (quanta 2/4/8), CFS [3], and Stride [18]. A MaskablePPO ablation without graph structure is included to estimate the contribution of the graph representation. SRTF with oracle access to future burst lengths serves as a theoretical upper bound.

Training. 150 000 timesteps, learning rate 3×10^{-4}, n_{steps}=2048, clip ratio ϵ=0.2. Multi-seed evaluation: 5 training seeds × 500 test workloads with bootstrap 95 % confidence intervals (5 000 resamples).

V. RESULTS

A. Full Benchmark

Table I has the numbers. On uniform workloads, our model achieves 18.54 average wait (95 % CI [17.81, 19.25]) versus 24.31 for CFS—a reduction of 23.7 %. Starvation drops from 87.3 % to 62.9 %, a difference of 24.4 percentage points.

TABLE III
MULTI-SEED EVALUATION (5 TRAINING SEEDS × 500 WORKLOADS).

Model	Avg Wait	95 % CI
Plain MaskablePPO	21.62	[21.20, 22.06]
GraphSAGE + MaskablePPO	20.96	[20.86, 21.07]
Paired diff. (G−P)	-0.67	$[-1.12, -0.26]$

Almost all configurations we tested outperform FCFS and Round Robin comfortably, but beating CFS by this margin was the real test.

Realistic workloads present a more nuanced picture. The wait-time advantage over CFS shrinks to roughly 1.2 % (28.00 vs. 28.33)—effectively a tie on throughput. Yet starvation decreases by 11.5 percentage points (67.7 % vs. 79.2 %). Beating CFS on raw efficiency is difficult when bursts follow exponential distributions; fairness is where our approach excels.

B. Contribution Decomposition

The paired ablation (Table II) reveals important patterns. We fed identical 500 workloads into both agents and used bootstrap resampling to compute paired differences.

On uniform workloads, RL by itself bridges roughly half the gap to the SRTF oracle (49 %). Adding the graph contributes another 8 %. Both improvements are statistically significant.

We were surprised by the realistic-workload result. Without the graph, the flat-MLP agent is actually slightly *worse* than CFS (paired diff $+0.6$; the CI straddles zero). With the graph, however, the sign changes: -0.7, and the CI excludes zero. In other words, on more challenging workloads, the graph is not merely useful—without it, there is no significant improvement over CFS at all.

C. Multi-Seed Robustness

Five models using different random seeds were trained separately, and each was tested on 500 workloads. The pairwise difference between GraphSAGE and the flat MLP comes out to -0.67 ticks (95 % CI $[-1.12, -0.26]$)—small but highly consistent across all repetitions (Table III).

D. Visualisation

An interactive Streamlit dashboard was created for qualitative analysis of scheduling behaviour. Fig. 3 illustrates the per-process state table and Gantt chart; Fig. 4 presents the dynamic process graph with all three edge families.

VI. DISCUSSION

CFS proved more competitive than initially anticipated on realistic workloads, since its virtual-runtime weighting handles heterogeneous exponential bursts well enough that a flat MLP cannot extract further leverage. What the graph brings is relational context: by analysing burst-history edges, the policy detects process clusters exhibiting similar behaviour and uses that as a rough proxy for predicting which ones will finish

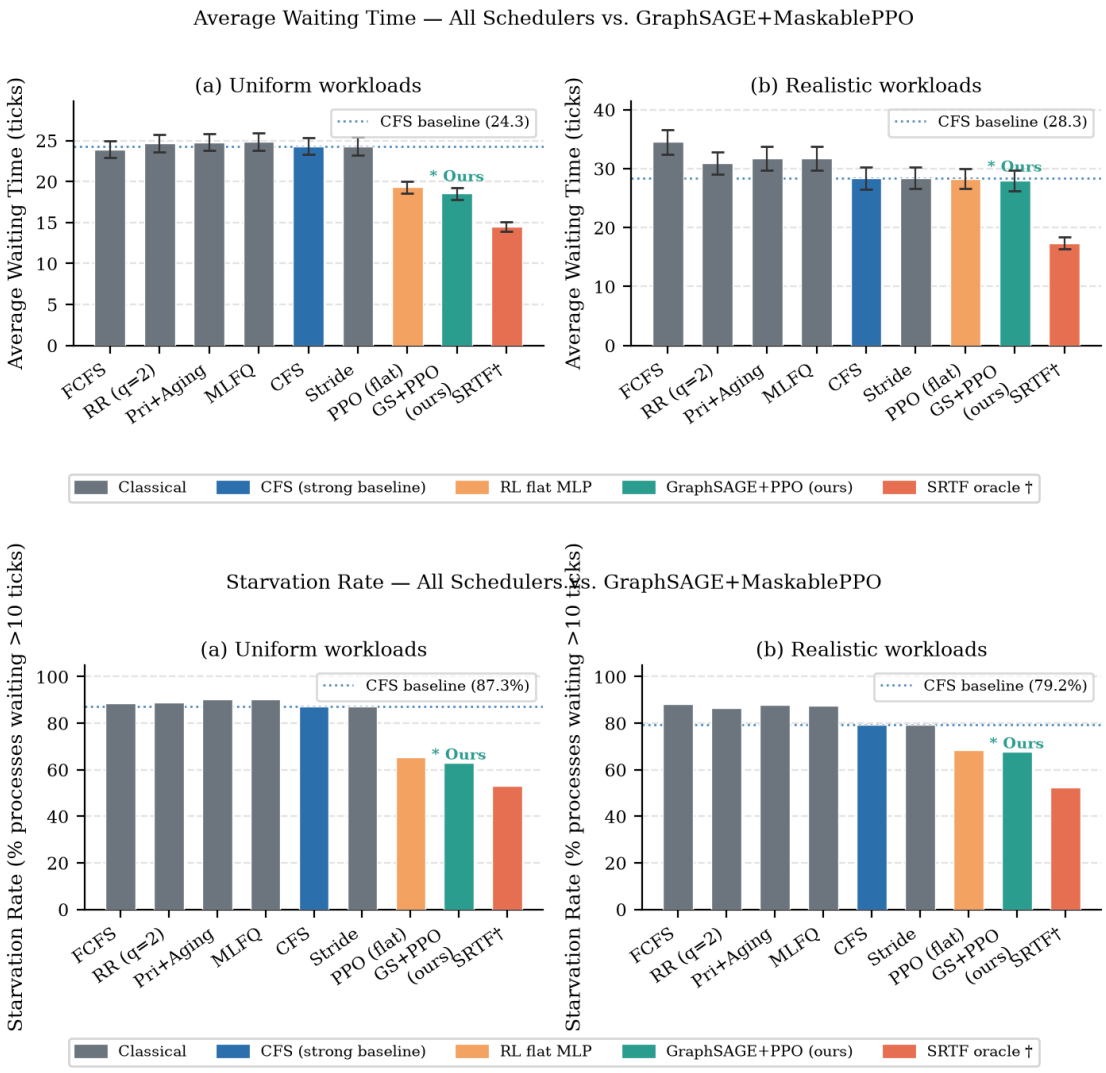


Fig. 2. Top: average waiting time across all schedulers (500 episodes, bootstrap 95 % CIs). Bottom: starvation rate. CFS baseline indicated by blue dotted line. The proposed model achieves the lowest non-oracle waiting time on uniform workloads and reduces starvation by 24.4 pp (uniform) and 11.5 pp (realistic) relative to CFS. SRTF utilises hidden burst lengths and serves as a theoretical lower bound.

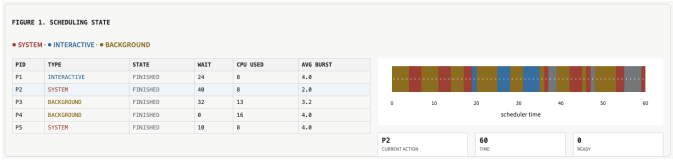


Fig. 3. Dashboard view of GraphSAGE + MaskablePPO on seed 42 with 5 processes. Left: per-process scheduling state. Right: Gantt timeline spanning 60 ticks. P2 (SYSTEM) accumulates the highest waiting time due to repeated I/O blocking; P4 (BACKGROUND) receives priority upon exceeding the aging threshold.

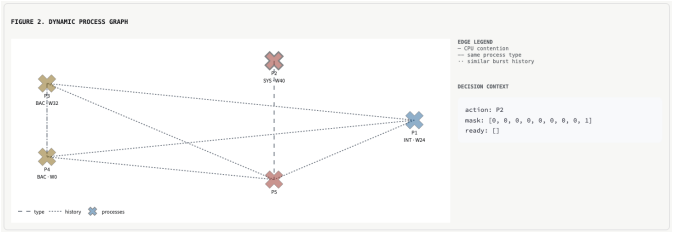


Fig. 4. Dynamic process graph as visualised in the dashboard. Edge families: CPU contention (solid), type similarity (dashed), and burst-history similarity (dotted). The graph is reconstructed at every tick and aggregated by GraphSAGE into a 48-dimensional policy input.

quickly. It amounts to a learned, approximate version of SRTF-style reasoning, built from observable history instead of hidden future information.

The remaining gap to SRTF on realistic workloads—10.9 ticks in the paired analysis (Table II)—is, in our view, mostly information-theoretic. Since the coefficient of variation for exponential distributions equals 1.0, the next burst could be several times larger or smaller than the average. No policy

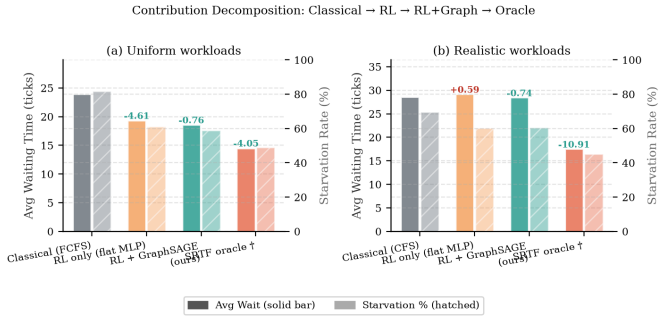


Fig. 5. Contribution decomposition (Classical → RL → RL+Graph → SRTF). Solid bars: average waiting time. Hatched bars: starvation rate. On realistic workloads, RL alone does not significantly outperform CFS (+0.6, n.s.), but the addition of the graph yields a significant improvement (−0.7).

without the ability to see the future can close that gap. On uniform workloads, where variance is lower, the gap shrinks to 4.05 ticks.

If there is one major lesson, it is the starvation numbers. Across every condition we tested, 12–24 percentage points fewer processes starve compared to CFS. The aging penalty (Eq. 1) gives the policy a strong push to attend to processes that have been waiting too long. To our knowledge, no other RL-based scheduler learns anti-starvation behaviour purely from reward shaping.

A. Limitations

The entire system operates in a Python simulator—we have not yet touched the kernel. The fixed observation size limits us to 8 processes. Our "realistic" workload distributions are based on textbook parameters, not actual `/proc` traces from production machines. And all three edge types share the same mean-pooling aggregation; relation-specific kernels like R-GCN [19] may yield further gains.

VII. Conclusion

The takeaway is twofold. First, starvation decreases by 12–24 percentage points compared to CFS in every condition tested—the learned aging reward works. Second, the graph is not optional. On realistic workloads—the ones that actually resemble what a real OS would face—a flat RL agent cannot outperform CFS. The relational structure is what provides the signal to recognise scheduling opportunities that a flat observation vector misses entirely.

Next steps: eBPF [20] integration into a real Linux kernel, variable-length process queues with sparse graph attention, and evaluation on production workload traces.

References

[1] W. L. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 1024–1034.

[2] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021, maskablePPO from the sb3-contrib package: https://sb3-contrib.readthedocs.io.

[3] I. Molnar, "Completely fair scheduler," Linux Kernel Documentation, 2007, [Online]. Available: https://www.kernel.org/doc/html/latest/scheduler/sched-design-CFS.html.

[4] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, 1st ed. Arpaci-Dusseau Books, 2023, [Online]. Available: https://pages.cs.wisc.edu/~remzi/OSTEP/.

[5] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018.

[6] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in *Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets)*. ACM, 2016.

[7] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, "Learning scheduling algorithms for data processing clusters," in *Proceedings of ACM SIGCOMM*. ACM, 2019.

[8] D. Zhang, D. Dai, Y. He, F. S. Bao, and B. Xie, "RLScheduler: An automated HPC batch job scheduler using reinforcement learning," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. IEEE, 2020.

[9] C. Zhang, W. Song, Z. Cao, J. Zhang, P. S. Tan, and C. Xu, "Learning to dispatch for job shop scheduling via deep reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1621–1632.

[10] I. Echeverria, M. Murua, and R. Santana, "A review of graph neural networks for job shop scheduling problems," arXiv preprint arXiv:2406.14096, 2024.

[11] E. N. Adimora *et al.*, "Deep reinforcement learning for efficient HPC job scheduling," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. IEEE, 2024.

[12] S. Huang and S. Ontañón, "A closer look at invalid action masking in policy gradient algorithms," in *Proceedings of the Florida Artificial Intelligence Research Society Conference (FLAIRS)*, vol. 35, 2022.

[13] S. Lassoued, S. Lier, and A. Schwung, "Policy-based reinforcement learning with action masking for dynamic job shop scheduling under uncertainty: Handling random arrivals and machine failures," arXiv preprint arXiv:2601.09293, 2026.

[14] A. Mamirov, "Reducing fragmentation and starvation in GPU clusters through dynamic multi-objective scheduling," arXiv preprint arXiv:2512.10980, 2025.

[15] J. Lee, S. Jung, and H. Jo, "STUN: Reinforcement-learning-based optimization of kernel scheduler parameters," *Applied Sciences*, vol. 12, no. 14, p. 7072, 2022.

[16] M. Towers *et al.*, "Gymnasium," https://gymnasium.farama.org, 2023.

[17] E. G. Coffman and L. Kleinrock, "Feedback queueing models for time-shared systems," *Journal of the ACM*, vol. 15, no. 4, pp. 549–576, 1968.

[18] C. A. Waldspurger and W. E. Weihl, "Stride scheduling: Deterministic proportional-share resource management," MIT, Tech. Rep. MIT/LCS/TM-528, 1995.

[19] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in *Proceedings of the European Semantic Web Conference (ESWC)*. Springer, 2018.

[20] Linux Foundation, "eBPF – extended Berkeley packet filter," https://ebpf.io, 2024.